FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name:	Diaz, James Daniel M.
Section:	AN42
Professor:	Crisola G. Tan
Date:	October 5, 2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.

5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:
   - Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
   - Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:
   - MS Word, Google Docs, or LibreOffice
   - Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:
   - Knowledge Representation & Reasoning
   - Case-Based Reasoning cycle
   - Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
   - Google Drive for shared documentation
   - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair
Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study
Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:
Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base
Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment
Your program must include a way to measure similarity between the new case and past cases.
Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✓ Similarity score or ranking
✓ The most relevant past case retrieved

5. Adapt the solution
Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✓ "New case saved" or similar confirmation

✓ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than

rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with “theft involving electronic transfer.” System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:

After final verdict, the case is stored—enhancing the system’s knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.

Student submits a math solution with repeated errors.

System retrieves past students with similar error patterns (e.g., misusing formulas).

It adapts feedback and personalized lessons based on successful interventions.

Learning:

System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks

Case 1: Disaster Response Decision Support System

```
import json
import os

CASE_FILE = "case_base.json"


# ============================================================
# INITIAL CASE BASE
# ============================================================

INITIAL_CASES = [
  {
    "name": "Typhoon Ursula",
    "year": 2019,
    "disaster_type": "Typhoon",
    "hazard": "Flooding",
    "severity": 8,
```

```
    "population": 70000,
    "rainfall": 280,
    "shelters": 15,
    "evacuation": "Pre-emptive evacuation of low-lying and coastal areas",
    "resources": "Rescue boats, food, water, medicine and emergency personnel",
    "shelter_strategy": "Use existing evacuation centers and schools",
    "outcome": "70,000 residents evacuated successfully with minimal casualties."
  },
  {
    "name": "Typhoon Odette",
    "year": 2021,
    "disaster_type": "Typhoon",
    "hazard": "Flooding",
    "severity": 9,
    "population": 90000,
    "rainfall": 310,
    "shelters": 20,
    "evacuation": "Early evacuation of highly vulnerable communities",
    "resources": "Emergency food, water, generators, rescue teams and medical supplies",
    "shelter_strategy": "Open evacuation centers and prepare temporary shelters",
    "outcome": "Most affected residents were evacuated before severe conditions."
  },
  {
    "name": "Typhoon Ulysses",
    "year": 2020,
    "disaster_type": "Typhoon",
    "hazard": "Flooding",
    "severity": 8,
    "population": 85000,
    "rainfall": 300,
    "shelters": 18,
    "evacuation": "Evacuate residents near rivers and flood-prone communities",
    "resources": "Rescue boats, relief goods, medical teams and communication equipment",
    "shelter_strategy": "Prioritize high-capacity evacuation centers",
    "outcome": "Large-scale evacuation was completed, although several shelters reached capacity."
  },
  {
```

```
        "name": "Typhoon Rai",
        "year": 2021,
        "disaster_type": "Typhoon",
        "hazard": "Strong Winds",
        "severity": 9,
        "population": 60000,
        "rainfall": 220,
        "shelters": 14,
        "evacuation": "Evacuate residents from unsafe structures before landfall",
        "resources": "Generators, food, water, medical supplies and emergency crews",
        "shelter_strategy": "Use reinforced evacuation centers",
        "outcome": "Evacuation was completed before landfall and emergency supplies were distributed."
    },
    {
        "name": "Metro Manila Flood",
        "year": 2022,
        "disaster_type": "Flood",
        "hazard": "Urban Flooding",
        "severity": 7,
        "population": 80000,
        "rainfall": 260,
        "shelters": 17,
        "evacuation": "Evacuate residents from low-lying urban barangays",
        "resources": "Rescue boats, pumps, food, water and medical teams",
        "shelter_strategy": "Distribute evacuees across multiple urban shelters",
        "outcome": "Flooding was controlled and residents were distributed across available shelters."
    }
]


# ============================================================
# FILE HANDLING
# ============================================================

def load_cases():
    """Load the case base from JSON."""
    if not os.path.exists(CASE_FILE):
        save_cases(INITIAL_CASES)
        return INITIAL_CASES.copy()

    try:
```

```
        with open(CASE_FILE, "r", encoding="utf-8") as file:
            return json.load(file)
    except (json.JSONDecodeError, OSError):
        print("Warning: Could not read case base.")
        print("Restoring the original case base.\n")
        save_cases(INITIAL_CASES)
        return INITIAL_CASES.copy()


def save_cases(cases):
    """Save the case base to JSON."""
    with open(CASE_FILE, "w", encoding="utf-8") as file:
        json.dump(cases, file, indent=4)


# ============================================================
# INPUT FUNCTIONS
# ============================================================

def get_int(prompt, minimum=0):
    """Safely obtain an integer."""
    while True:
        try:
            value = int(input(prompt))

            if value < minimum:
                print(f"Please enter a value of at least {minimum}.")
                continue

            return value

        except ValueError:
            print("Please enter a valid number.")


def get_new_problem():
    """Collect the new disaster problem."""

    print("\n" + "=" * 60)
    print("[NEW PROBLEM DESCRIPTION]")
    print("=" * 60)

    print("\nEnter the characteristics of the new disaster.")
```

```
disaster_type = input(
    "Disaster type [Typhoon/Flood/Earthquake]: "
).strip()

hazard = input(
    "Expected hazard [Flooding/Urban Flooding/Strong Winds/etc.]: "
).strip()

severity = get_int(
    "Severity (1-10): ",
    1
)

while severity > 10:
    print("Severity must be between 1 and 10.")
    severity = get_int("Severity (1-10): ", 1)

population = get_int(
    "Expected affected population: ",
    0
)

rainfall = get_int(
    "Expected rainfall (mm): ",
    0
)

shelters = get_int(
    "Available evacuation shelters: ",
    0
)

return {
    "name": "New Disaster Event",
    "year": 2026,
    "disaster_type": disaster_type,
    "hazard": hazard,
    "severity": severity,
    "population": population,
    "rainfall": rainfall,
    "shelters": shelters
}
```

```
# ============================================================
# CBR RETRIEVAL
# ============================================================

def similarity_score(new_case, old_case):
    """
    Calculate similarity between two cases.

    Total score = 100%

    Disaster type : 20%
    Hazard        : 20%
    Severity      : 15%
    Population    : 15%
    Rainfall      : 15%
    Shelters      : 15%
    """

    score = 0

    # Disaster type
    if new_case["disaster_type"].lower() == old_case["disaster_type"].lower():
        score += 20

    # Hazard
    if new_case["hazard"].lower() == old_case["hazard"].lower():
        score += 20
    else:
        # Partial match for related hazards
        new_hazard = new_case["hazard"].lower()
        old_hazard = old_case["hazard"].lower()

        if "flood" in new_hazard and "flood" in old_hazard:
            score += 12

        elif "wind" in new_hazard and "wind" in old_hazard:
            score += 12

    # Severity similarity
    severity_difference = abs(
        new_case["severity"] - old_case["severity"]
    )

    severity_similarity = max(
```

```
        0,
        1 - (severity_difference / 10)
    )

    score += severity_similarity * 15

    # Population similarity
    if new_case["population"] > 0 and old_case["population"] > 0:
        population_difference = abs(
            new_case["population"] - old_case["population"]
        ) / max(
            new_case["population"],
            old_case["population"]
        )

        population_similarity = max(
            0,
            1 - population_difference
        )

        score += population_similarity * 15

    # Rainfall similarity
    if new_case["rainfall"] > 0 and old_case["rainfall"] > 0:
        rainfall_difference = abs(
            new_case["rainfall"] - old_case["rainfall"]
        ) / max(
            new_case["rainfall"],
            old_case["rainfall"]
        )

        rainfall_similarity = max(
            0,
            1 - rainfall_difference
        )

        score += rainfall_similarity * 15

    # Shelter similarity
    if new_case["shelters"] > 0 and old_case["shelters"] > 0:
        shelter_difference = abs(
            new_case["shelters"] - old_case["shelters"]
        ) / max(
            new_case["shelters"],
```

```
            old_case["shelters"]
        )

        shelter_similarity = max(
            0,
            1 - shelter_difference
        )

        score += shelter_similarity * 15

    return score


def retrieve_cases(new_case, cases):
    """Retrieve and rank similar cases."""

    ranked_cases = []

    for case in cases:
        score = similarity_score(new_case, case)

        ranked_cases.append({
            "case": case,
            "score": score
        })

    ranked_cases.sort(
        key=lambda item: item["score"],
        reverse=True
    )

    return ranked_cases


# ============================================================
# DISPLAY RETRIEVAL RESULTS
# ============================================================

def display_retrieval(ranked_cases):
    """Display similarity rankings."""

    print("\n" + "=" * 60)
    print("[RETRIEVAL PHASE]")
    print("=" * 60)
```

```
    print(
        "\nThe system is retrieving cases with characteristics "
        "similar to the new disaster..."
    )

    print(
        "\nDECLARATION: Similar cases have been retrieved "
        "from the case base."
    )

    print("\n[RANKED SIMILAR CASES]")
    print("-" * 60)

    for index, item in enumerate(ranked_cases, start=1):

        case = item["case"]
        score = item["score"]

        print(
            f"{index}. {case['name']} ({case['year']}) "
            f"- Similarity: {score:.2f}%"
        )

    print("-" * 60)


# ============================================================
# ADAPTATION
# ============================================================

def adapt_solution(new_case, retrieved_cases):
    """
    Adapt recommendations from the most similar cases.

    The first three retrieved cases are used as evidence.
    """

    top_cases = retrieved_cases[:3]

    print("\n" + "=" * 60)
    print("[REUSE AND ADAPTATION PHASE]")
    print("=" * 60)
```

```
print(
    "\nThe system is adapting solutions from the most "
    "similar previous cases."
)

# ------------------------------------------------------
# Evacuation
# ------------------------------------------------------

if "flood" in new_case["hazard"].lower():

    evacuation = (
        "Begin pre-emptive evacuation of low-lying and "
        "flood-prone urban areas. Prioritize residents "
        "near rivers, drainage channels and other high-risk zones."
    )

else:

    evacuation = (
        "Begin evacuation of residents from areas identified "
        "as unsafe based on the expected disaster conditions."
    )

# ------------------------------------------------------
# Resources
# ------------------------------------------------------

if new_case["population"] >= 80000:

    resources = (
        "Deploy additional rescue teams and pre-position "
        "food, drinking water, medicine, generators and "
        "emergency communication equipment."
    )

else:

    resources = (
        "Pre-position food, drinking water, medicine, "
        "generators and emergency response personnel."
    )

# ------------------------------------------------------
```

```
# Shelters
# ------------------------------------------------------

if new_case["shelters"] < 15:

    shelter_strategy = (
        "Available shelter capacity may be insufficient. "
        "Prepare temporary shelters and coordinate with "
        "nearby communities for additional evacuation space."
    )

elif new_case["population"] / new_case["shelters"] > 5000:

    shelter_strategy = (
        "Activate all available shelters and prepare "
        "additional temporary shelters because expected "
        "population per shelter is high."
    )

else:

    shelter_strategy = (
        "Activate available evacuation centers and "
        "distribute evacuees across facilities."
    )

# ------------------------------------------------------
# Show which cases influenced the solution
# ------------------------------------------------------

print("\nCases used as solution evidence:")

for item in top_cases:
    print(
        f"- {item['case']['name']} "
        f"({item['score']:.2f}% similarity)"
    )

return {
    "evacuation": evacuation,
    "resources": resources,
    "shelter_strategy": shelter_strategy
}
```

```
# ============================================================
# FINAL SOLUTION
# ============================================================

def display_solution(solution):
    """Display the adapted final solution."""

    print("\n" + "=" * 60)
    print("[FINAL SOLUTION]")
    print("=" * 60)

    print("\nRecommended Disaster Response:\n")

    print("1. EVACUATION STRATEGY")
    print(f"   {solution['evacuation']}")

    print("\n2. RESOURCE ALLOCATION")
    print(f"   {solution['resources']}")

    print("\n3. SHELTER CAPACITY")
    print(f"   {solution['shelter_strategy']}")

    print("\nThe recommendations above were adapted from "
          "previous disaster cases.")


# ============================================================
# REVISION
# ============================================================

def collect_outcome():
    """Collect the actual outcome after executing the solution."""

    print("\n" + "=" * 60)
    print("[REVISION PHASE]")
    print("=" * 60)

    print(
        "\nAfter the recommended response has been executed, "
        "enter the actual outcome."
    )

    outcome = input(
```

```
        "\nDescribe the actual outcome: "
    ).strip()

    while not outcome:
        print("Outcome cannot be empty.")
        outcome = input(
            "Describe the actual outcome: "
        ).strip()

    return outcome


# ============================================================
# RETENTION
# ============================================================

def retain_case(
    cases,
    new_case,
    solution,
    outcome
):
    """Add the new experience to the case base."""

    new_case["name"] = "Retained Case - Urban Disaster Response"
    new_case["year"] = 2026

    new_case["evacuation"] = solution["evacuation"]
    new_case["resources"] = solution["resources"]
    new_case["shelter_strategy"] = solution["shelter_strategy"]
    new_case["outcome"] = outcome

    cases.append(new_case)

    save_cases(cases)

    print("\n" + "=" * 60)
    print("[RETAIN PHASE]")
    print("=" * 60)

    print(
        "\nNew case has been added to the case base."
    )
```

```
    print(
        "This experience can now be retrieved as evidence "
        "for future disaster-response decisions."
    )


# ============================================================
# DISPLAY CASE BASE
# ============================================================

def display_case_base(cases):
    """Display the complete case base."""

    print("\n" + "=" * 60)
    print("[CASE BASE]")
    print("=" * 60)

    print(
        f"\nTotal cases currently stored: {len(cases)}\n"
    )

    for index, case in enumerate(cases, start=1):

        print(f"Case {index}: {case['name']} ({case['year']})")
        print(f"  Disaster Type : {case['disaster_type']}")
        print(f"  Hazard        : {case['hazard']}")
        print(f"  Severity      : {case['severity']}/10")
        print(f"  Population    : {case['population']:,}")
        print(f"  Rainfall      : {case['rainfall']} mm")
        print(f"  Shelters      : {case['shelters']}")
        print(f"  Evacuation    : {case.get('evacuation', 'N/A')}")
        print(f"  Resources     : {case.get('resources', 'N/A')}")
        print(
            f"  Shelter Plan  : "
            f"{case.get('shelter_strategy', 'N/A')}"
        )
        print(f"  Outcome       : {case.get('outcome', 'N/A')}")
        print("-" * 60)


# ============================================================
# MAIN PROGRAM
# ============================================================
```

```
def main():

    cases = load_cases()

    print("\n" + "=" * 60)
    print(" CBR-BASED DISASTER RESPONSE DECISION SUPPORT SYSTEM")
    print("=" * 60)

    print(
        "\nCBR Cycle:"
        "\n  RETRIEVE -> REUSE/ADAPT -> REVISE -> RETAIN"
    )

    # ----------------------------------------------------
    # Step 1: New problem
    # ----------------------------------------------------

    new_case = get_new_problem()

    print("\n" + "=" * 60)
    print("[NEW PROBLEM DESCRIPTION]")
    print("=" * 60)

    print(f"Disaster Type       : {new_case['disaster_type']}")
    print(f"Expected Hazard     : {new_case['hazard']}")
    print(f"Severity            : {new_case['severity']}/10")
    print(
        f"Affected Population: "
        f"{new_case['population']:,}"
    )
    print(f"Expected Rainfall   : {new_case['rainfall']} mm")
    print(f"Available Shelters  : {new_case['shelters']}")

    # ----------------------------------------------------
    # Step 2: Retrieve
    # ----------------------------------------------------

    ranked_cases = retrieve_cases(
        new_case,
        cases
    )

    display_retrieval(ranked_cases)
```

```
# ----------------------------------------------------
# Step 3: Reuse / Adapt
# ----------------------------------------------------

solution = adapt_solution(
    new_case,
    ranked_cases
)

display_solution(solution)

# ----------------------------------------------------
# Step 4: Revise
# ----------------------------------------------------

outcome = collect_outcome()

# ----------------------------------------------------
# Step 5: Retain
# ----------------------------------------------------

retain_case(
    cases,
    new_case,
    solution,
    outcome
)

# ----------------------------------------------------
# Step 6: Display updated case base
# ----------------------------------------------------

display_case_base(cases)

print("\n" + "=" * 60)
print("CBR CYCLE COMPLETE")
print("=" * 60)

print(
    "\nRETRIEVE -> REUSE/ADAPT -> REVISE -> RETAIN"
)

print(
    "\nThe new experience has become part of the "
```

```
        "system's future case evidence."
    )


# ============================================================
# PROGRAM START
# ============================================================

if __name__ == "__main__":
    main()
```